

COMP3711: Design and Analysis of Algorithms

WU Jinjun

September 7, 2026

1 - Asymptotic

O, Ω, Θ are used to describe the asymptotic behavior of functions.

- (a) $f(n) = O(g(n))$ means that there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$.
- (b) $f(n) = \Omega(g(n))$ means that there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n \geq n_0$, $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} > 0$.
- (c) $f(n) = \Theta(g(n))$ means that there exist positive constants c_1, c_2 and n_0 such that $c_1g(n) \leq f(n) \leq c_2g(n)$ for all $n \geq n_0$, $0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$.
- (d) Not Comparable: $f(n)$ and $g(n)$ are not comparable if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ does not exist.

Asymptotic Examples

constant < logarithmic < polynomial < exponential

$O(1) \subset O(\log \log n) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^k) \subset O(2^n) \subset O(n!) \subset O(n^n)$

2 - Divide and Conquer (D&C)

Main idea of D&C

Solve a problem of size n by breaking it into one or more smaller problems of size less than n . Solve the smaller problems recursively and combine their solutions to solve the large problem.

Example 1: Binary Search

Algorithm 1 Binary Search

```
function BINARYSEARCH( $A, x$ )  
   $low \leftarrow 0$   
   $high \leftarrow \text{length}(A) - 1$   
  while  $low \leq high$  do  
     $mid \leftarrow \left\lfloor \frac{low + high}{2} \right\rfloor$   
    if  $A[mid] = x$  then  
      return  $mid$   
    else if  $A[mid] < x$  then  
       $low \leftarrow mid + 1$   
    else  
       $high \leftarrow mid - 1$   
    end if  
  end while  
  return -1  
end function
```

Analysis of Binary Search $T(n)$ is the number of comparisons made by the binary search algorithm on an array of size n .

Recurrence relation: $T(n) \leq T(\frac{n}{2}) + 2$, basis: $T(1) = T(0) = 1$

Solve the recurrence relation: $T(n) \leq T(\frac{n}{2}) + 2 \leq T(\frac{n}{4}) + 4 \leq \dots \leq T(1) + 2\log_2 n = 1 + 2\log_2 n$

So $T(n) = O(\log n)$

Exercise 1.1: Rotated Sorted Array

Let $A[1..n]$ be a sorted array of n distinct numbers that has been rotated $n - k$ steps for some unknown integer $k \in [1, n - 1]$. That is, $A[1..k]$ is sorted in increasing order, and $A[k+1..n]$ is also sorted in increasing order, and $A[n] < A[1]$. The following array A is an example of $n = 16$ elements with $k = 10$.

$A = [9, 13, 16, 18, 19, 23, 28, 31, 37, 42, 0, 1, 2, 5, 7, 8]$.

(1) Design an $O(\log n)$ -time algorithm to find the value of k . ($A[k]$ is the maximum element in the array)

Algorithm 2 Find Rotated Index

```
function FINDROTATEDINDEX( $A, l, r$ )  
     $mid \leftarrow \left\lfloor \frac{l+r}{2} \right\rfloor$   
    if  $A[mid] > A[mid+1]$  then  
        return  $mid$   
    else if  $A[mid] \geq A[l]$  then  
        return FINDROTATEDINDEX( $A, mid+1, r$ )  
    else  
        return FINDROTATEDINDEX( $A, l, mid-1$ )  
    end if  
end function
```

(2) Design an $O(\log n)$ -time algorithm that for any given x , find x in the rotated sorted array, or report that it does not exist.

Algorithm 3 Search in Rotated Sorted Array

```
function SEARCHROTATED( $A, x, n$ )  
     $k \leftarrow$  FINDROTATEDINDEX( $A, 1, n$ )  
    if  $x \geq A[1]$  then  
        return BINARYSEARCH( $A[1..k], x$ )  
    else  
        return BINARYSEARCH( $A[k+1..n], x$ )  
    end if  
end function
```

Exercise 1.2: Finding the Last 0

You are given an array $A[1..n]$ of n elements, each of which is either 0 or 1. The array contains a sequence of 0s followed by a sequence of 1s.

(1) Design an $O(\log n)$ -time algorithm to find the position k of the last 0 in the array. If there is no 0 in the array, return -1.

Algorithm 4 Find Last 0

```
function FINDLAST0( $A, l, r$ )  
     $mid \leftarrow \left\lfloor \frac{l+r}{2} \right\rfloor$   
    if  $A[mid] = 0$  and  $A[mid+1] = 1$  then  
        return  $mid$   
    else if  $A[mid] = 0$  then  
        return FINDLAST0( $A, mid+1, r$ )  
    else  
        return FINDLAST0( $A, l, mid-1$ )  
    end if  
end function
```

(2) Suppose that k is much smaller than n . Design an $O(\log k)$ -time algorithm to find the position k of the last 0 in the array.

Algorithm 5 Find Last 0 with $O(\log k)$

```

 $i \leftarrow 1$ 
while  $i < n \wedge A[i] = 0$  do
     $i \leftarrow \min\{2i, n\}$ 
end while
if  $A[i] = 0$  then
    return  $i$   $\triangleright$  all zeros: last 0 is  $n$ 
else if  $i = 1$  then
    return  $-1$   $\triangleright$  no zero
else
    return FINDLAST0( $A, \lfloor i/2 \rfloor, i$ )
end if

```

Example 2: Merge Sort

Review of Hanoi Tower Problem

Problem: There are three pegs, and n disks of different sizes. The disks are initially stacked on one peg in decreasing size from bottom to top. The goal is to move the entire stack to another peg, obeying the following rules:

- (a) Only one disk can be moved at a time.
- (b) Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty peg.
- (c) No disk may be placed on top of a smaller disk.

Solution: Let $T(n)$ be the minimum number of moves required to transfer n disks from one peg to another.

- (a) $T(1) = 1$.
- (b) To move n disks from peg A to peg C, we first move the top $n - 1$ disks from A to B, then move the largest disk from A to C, and finally move the $n - 1$ disks from B to C.
So $T(n) = 2T(n - 1) + 1$.
- (c) Solve the recurrence relation: $T(n) = 2T(n - 1) + 1 = 2(2T(n - 2) + 1) + 1 = 4T(n - 2) + 3 = 4(2T(n - 3) + 1) + 3 = 8T(n - 3) + 7 = \dots = 2^k T(n - k) + (2^k - 1)$.
Let $k = n - 1$, we have $T(n) = 2^{n-1}T(1) + (2^{n-1} - 1) = 2^n - 1$.

Algorithm 6 Hanoi Tower

```
function HANOI( $n, A, B, C$ )  
  if  $n = 1$  then  
    Move disk from  $A$  to  $C$   
  else  
    HANOI( $n - 1, A, C, B$ )  
    Move disk from  $A$  to  $C$   
    HANOI( $n - 1, B, A, C$ )  
  end if  
end function
```
